Once the build or pipeline is executed successfully, you can go to the SonarQube portal and verify the recently analysed project.

Verifying the bugs, vulnerabilities and code smells

There are different tools for different languages:

- Java, .NET, JavaScript, etc: SonarQube
- Android: Lint
- iOS: SwiftLint, OCLint

Code analysis tools can be integrated in the automation pipeline using plugins or the command line.

Continuous integration (CI)

In the traditional organisational culture, manual activities are paramount in the management of the application's life cycle. Organisations, project teams and business units face many challenges, as shown in Figure 3.

What is the immediate solution to address the challenges mentioned in Figure 3? The answer is continuous integration (CI), which is one of the main DevOps practices and a pillar of this culture's transformation process. Each check-in from a developer can trigger a build job or pipeline that performs automated build, unit test execution and package creation. CI helps to detect issues in the code, based on compilation and unit test execution, at an early stage. The faster you fail, the faster you recover.

The objective of automation is to make existing practices and processes more effective in a way that communication and collaboration between teams becomes better.

Figure 4 shows some of the best practices to make CI more effective.

Let's understand, from Figure 5, how CI flows. We can also consider it as outlining the following steps.

Create a *Build Job* in Jenkins and configure the command *mvn clean package*. Before executing this command, configure Maven, JDK and Git in *Manage Jenkins -> Global Tools Configuration.*

The following is a console log of a Jenkins job that executes a sample Java application. Click on *Build Now* in the Jenkins dashboard.

```
Started by user admin
Building in workspace F:\1.DevOps\2018\JenkinsHomeTemplate\
sharedspace
No credentials specified
 > git.exe rev-parse --is-inside-work-tree # timeout=10
Fetching changes from the remote Git repository
 > git.exe config remote.origin.url https://github.com/
mitesh51/spring-petclinic.git # timeout=10
Fetching upstream changes from https://github.com/mitesh51/
spring-petclinic.git
.
.
.
```

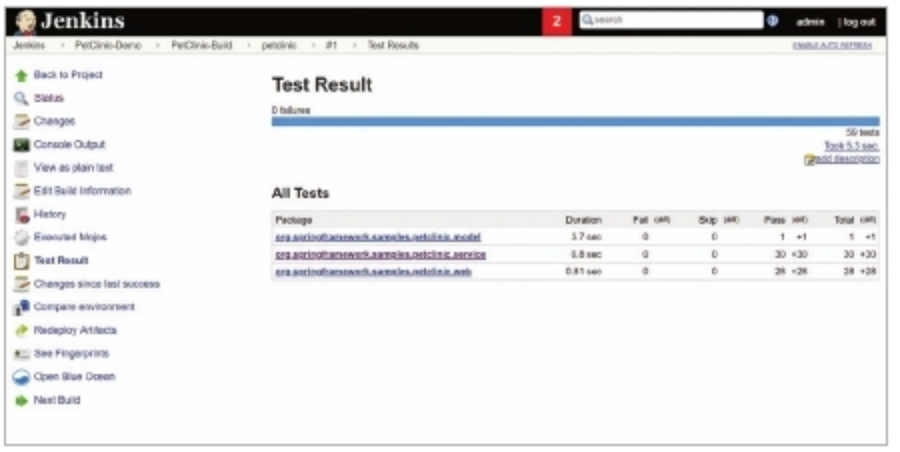

Figure 6: CI - Unit test results

```
Parsing POMs
Established TCP socket on 53326
[sharedspace] $ "C:\Program Files (x86)\Java\jdk1.8.0_192/
bin/java" -cp F:\1.DevOps\2018\JenkinsHomeTemplate\plugins\
maven-plugin\WEB-INF\lib\maven35-agent-1.12.jar;F:\1.
DevOps\2018\apache-maven-3.5.3\boot\plexus-classworlds-
2.5.2.jar;F:\1.DevOps\2018\apache-maven-3.5.3/conf/logging
jenkins.maven3.agent.Maven35Main F:\1.DevOps\2018\apache-
maven-3.5.3 F:\1.DevOps\2018\JenkinsHomeTemplate\war\WEB-INF\
lib\remoting-3.29.jar F:\1.DevOps\2018\JenkinsHomeTemplate\
plugins\maven-plugin\WEB-INF\lib\maven35-interceptor-1.12.jar
F:\1.DevOps\2018\JenkinsHomeTemplate\plugins\maven-plugin\
WEB-INF\lib\maven3-interceptor-commons-1.12.jar 53326
<===[JENKINS REMOTING CAPACITY]===>channel started
Executing Maven: -B -f F:\1.DevOps\2018\JenkinsHomeTemplate\
sharedspace\pom.xml clean package
[INFO] Scanning for projects...
[HUDSON] Collecting dependencies info
[INFO]
[INFO] ------------< org.springframework.samples:spring-
petclinic >------------
[INFO] Building petclinic 4.2.5-SNAPSHOT
[INFO] --------------------------------[ war ]--------------
-----------------
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @
spring-petclinic ---
[INFO] Deleting F:\1.DevOps\2018\JenkinsHomeTemplate\
sharedspace\target
[INFO]
[INFO] --- cobertura-maven-plugin:2.7:clean (default) @
spring-petclinic ---
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-
resources) @ spring-petclinic ---
[INFO] Using 'UTF-8' encoding to copy filtered resources.
[INFO] Copying 18 resources
.
.
.
[INFO] --- maven-resources-plugin:2.6:testResources (default-
testResources) @ spring-petclinic ---
```